Prompts Compiled

San Marcos Group 1 (Tushig, Ananta, Carolina, Nick)

Using this calculator example I have given you (does not have to be as thorough) create a skeleton for this project (SRS) with more complex implementations to be left empty just as the file names themselves. Follow a Model-View-Controller structure. This skeleton should have the bare necessity features working such as the user interface and a basic API search using TMDB's API (here is my API KEY …).

For the csv dataset provided, using the existing ai search skeleton allow for backend query searches on the dataset using the exisiting columns as query. Basically allow for natural language processing into a sql query based off the columns. I have provided my open ai api key in the env file. Create test cases that for these as well that I can use curl on.

Fully implement and integrate the ai search feature now so that it takes what the user has input in the field and does an ai search (call to openai with a custom prompt and run a sql query on the dataset) if the user clicks that button.

I'm running into bugs when I try to use the AI search and recommendation features (500 responses from the backend when the NLP to SQL step fails). Go through both the frontend and backend, identify the root causes of these errors, and fix them. Add clear error handling and logging (console logs on the frontend and structured logs on the backend) so that failures return meaningful messages instead of crashing

Improve the algorithm so it's more precise and change it so it takes into account all the movies in the favorites list

Yeah, the recommendations are still very unrelated. Simplify the logic so it basically asks openai for the strongest common genre and keywords among the favorites and does a query on the dataset off that. Very simple but it will work.

Still having errors instead ask openai to say which one of these genres do the movies identify the most with "Drama

Comedy

Romance

Thriller

Action

Adventure

Crime

Family

Science Fiction

Fantasy" and then do a simple query on the dataset for the most popular movies with that genre and return those results.

Make these recommendations (the currently existing ones) about half width and on the left hand side. And now on the right hand side ask open ai directly for recommendations based on the movies in the favorites list and display them (6) on the right hand side. Maybe add a label that's dataset on the left and AI on the right.

Prompts

CRITICAL REQUIREMENTS

Frontend Experience, Accessibility, Full Internationalization

You are a senior full stack engineer working on a React plus Node.js movie discovery app that uses TMDB. You must implement a modern, accessible frontend experience that includes:

• A theme toggle with persisted user preference
• Search history with quick re run
• Strong accessibility improvements
• Full internationalization across the entire UI and all TMDB backed movie data (English, French, Spanish)

Everything must be additive. Do not break existing layout, theme, or search behavior.

1. STRATEGY
User Experience and Full I18N Pipeline

• Keep all existing pages and flows intact
• Add new features in a backward compatible way
• Use a centralized I18nContext for translations
• All UI labels must come from translations, no hard coded English
• TMDB data must be fetched in the selected language

2. FRONTEND
Theme Toggle with Persistence

Implement a light or dark mode toggle that:
• Updates the theme across the whole layout
• Persists user preference in localStorage
• Applies preference on first load without flashing
• Falls back to system or light mode if no preference is stored
• Does not break the current theme for users who never interact with it

3. FRONTEND
Search History and Quick Re Run

Build a proper search history panel that:
• Tracks recent search queries
• Stores them in memory and optionally localStorage
• Displays history in most recent order
• Allows clicking a past query to instantly run it again
• Includes a clear history option
• Is fully keyboard accessible and screen reader friendly

4. FRONTEND
Accessibility Improvements

Follow WCAG principles and ensure:
• All interactive elements are reachable by Tab and Shift Tab
• Logical focus order
• Correct focus management for modals, panels, dialogs
• ARIA roles and labels where needed
• All images and icons have alt text or fallbacks
• Colors meet contrast and legibility requirements

5. FRONTEND
Full Internationalization and Language Switch

Use an I18nContext that:
• Tracks the current language (en, fr, es)
• Provides t(key) for translated strings
• Provides setLanguage(lang)
• Persists language preference in localStorage

Translation dictionary must cover:
• Movie details labels (title, overview, release date, runtime, genres, cast, crew, rating)
• Section and carousel titles (Trending, Popular, Top rated, Recommended for you)

• Streaming labels (Available on, Rent, Buy, Stream)
• General UI labels (Search, Results, No results found, Load more)

Update all UI components so that nothing uses hard coded English.

6. BACKEND
TMDB Language Support

Update TmdbService:
• Every function accepts a language argument
• Append language=en or fr or es to TMDB calls
• Default to en when missing

Update controllers and routes to read language from:

1. Query parameter
2. Header such as x language or accept language
3. Fallback to en

Every movie request must pass the chosen language through to TMDB.

7. CLIENT SIDE FETCH
Language Aware API Calls

All frontend fetches must include language via query param or header, including:
• MovieDetails
• MovieCarouselRow
• WhereToWatch
• Any other movie data requests

8. MOVIEDETAILS UX
No Loading State on Language Switch

Behavior requirements:
• Initial mount can show a loading state
• After initial load, switching language must not show Loading
• Keep existing data on screen
• Fetch new language data in the background
• Update only localized fields when the response arrives

Use an isInitialLoading flag. Do not gate the whole component behind loading after first render.

9. VERIFICATION STEPS

Confirm that:
• Theme toggle works and persists
• Search history works, persists, and is usable

• Accessibility enhancements work
• All UI text comes from translations
• Switching languages updates UI labels and TMDB localized fields
• Persistence works across reloads
• MovieDetails updates smoothly without showing Loading
• Existing layout, search, and theming remain unchanged

AI SEARCH CAPABILITIES UI
(Formatted properly)

Add a UI component that clearly explains the AI search feature. Requirements:

• Component must be collapsible
• Positioned below the search bar
• Fully translated in all supported languages
• Uses I18nContext for title, description, examples header, and example text
• Accessible with ARIA attributes
• Works in dark and light themes
• No English text hard coded
• Integrated without breaking layout or search behavior

Example component name: AiSearchInfo.jsx

Translations must include:
• aiSearchInfo.title
• aiSearchInfo.description
• aiSearchInfo.examplesTitle
• And example queries

The design should match the AI Search button styling and follow the app's accessibility and theme rules.

Could you provide guidance on structuring the Favorites component so that it accurately retrieves user-saved data from the global context and renders it reliably within the UI?

Would you assist me in developing a modular structure for rendering recommended movies, ensuring that the logic is reusable and easy to maintain?

requesting assistance to ensure UI elements (e.g., "Your Favorites" header) properly respond to language selections like Spanish or French, emphasizing structured code placement guidance and minimal context-window clutter through isolated code blocks.

request for help making the “Your Favorites” header respond correctly to changes in language selection (English, Spanish, French). Required precise placement of translation hooks (useI18n), strategic code restructuring, and isolated formatting for readability

How do I hook Favorites into the language context so all the text changes when I switch languages?

Would you assist me in developing a modular structure for rendering recommended movies, ensuring that the logic is reusable and easy to maintain?

Could you assist me in determining why the Favorites component is rendering an empty view, even though data appears to be present in the context state?

Could you assist me in determining why the Favorites component is rendering an empty view, even though data appears to be present in the context state?

CRITICAL REQUIREMENTS - Additive Full-Stack Movie Carousels (Safety-Focused)

MANDATORY DIRECTIVE

You are an expert Full-Stack Developer proficient in React, Node.js, and Tailwind CSS.

CRITICAL: The previous attempt broke the UI (e.g., "black screen"). Your primary objective is safety and compatibility. You must ONLY ADD the new carousels below the existing content (shown in the image) and MUST NOT modify any existing styles, components, or layout (like the header or search bar).

1. STRATEGY: COMPONENT ISOLATION

CRITICAL: All new UI elements must be built inside new, isolated components.

MANDATORY: You will create ONE new parent component (e.g., <CarouselSection />).

MANDATORY: This <CarouselSection /> will be the ONLY new component added to the existing HomePage.jsx (or similar) file.

MANDATORY: This container can have its own dark background (e.g., bg-gray-900 or bg-black). This style MUST be applied only to this new container, not to the <body> or root App component.

CRITICAL: DO NOT modify any existing CSS files. All styling must be self-contained within the new components using Tailwind utility classes or, if necessary, a new, separate CSS file (e.g., MovieCarousel.css).

2. BACKEND REQUIREMENTS (Node.js)

CRITICAL: Create nine (9) new, non-conflicting API endpoints.

CRITICAL: Create the following GET routes in your Node.js server:

/api/movies/featured

/api/movies/new-releases

/api/movies/action

/api/movies/scifi

/api/movies/comedy

/api/movies/thriller

/api/movies/horror

/api/movies/suspense

/api/movies/drama

MANDATORY: For this task, each endpoint must return a mock JSON array of 15-20 movie objects. Each object must have this exact structure:

```
{
  "id": "tt123456",
  "title": "Movie Title Example",
  "posterUrl": "[https://placehold.co/300x450/222/FFF?text=Movie+Poster](https://placehold.co/300x450/222/FFF?text=Movie+Poster)"
}
```

3. FRONTEND REQUIREMENTS (React & Tailwind)

MANDATORY: 1. Create New Component: <MovieCarouselRow />

CRITICAL: Create a new file for this component (e.g., components/MovieCarouselRow.jsx).

MANDATORY: This component must accept title and fetchUrl as props.

MANDATORY: It will use useEffect to fetch data from its fetchUrl prop.

MANDATORY: It will render the row title (<h2>{title}</h2>) and a horizontal list of movie cards.

CRITICAL: This component MUST implement all 4 interaction models specified below.

MANDATORY: 2. Create New Component: <CarouselSection />

CRITICAL: Create a new file for this component (e.g., components/CarouselSection.jsx).

MANDATORY: This component will render the 9 <MovieCarouselRow /> components.

MANDATORY: Apply the dark background (e.g., bg-gray-900) and page padding (e.g., py-8 px-12) here, so it only affects the new content.

MANDATORY: This component will render the 9 rows:

```
<div className="bg-gray-900 py-8 px-12">
  <MovieCarouselRow title="Featured Movies" fetchUrl="/api/movies/featured" />
  <MovieCarouselRow title="New Releases" fetchUrl="/api/movies/new-releases" />
  <MovieCarouselRow title="Action" fetchUrl="/api/movies/action" />
  <MovieCarouselRow title="Sci-Fi" fetchUrl="/api/movies/scifi" />
  <MovieCarouselRow title="Comedy" fetchUrl="/api/movies/comedy" />
  <MovieCarouselRow title="Thriller" fetchUrl="/api/movies/thriller" />
  <MovieCarouselRow title="Horror" fetchUrl="/api/movies/horror" />
  <MovieCarouselRow title="Suspense" fetchUrl="/api/movies/suspense" />
  <MovieCarouselRow title="Drama" fetchUrl="/api/movies/drama" />
</div>
```

MANDATORY: 3. Modify Existing File: HomePage.jsx

CRITICAL: In this file, DO NOT DELETE OR MODIFY ANY EXISTING CODE.

MANDATORY: Find the main return (...) block. AFTER the existing search bar and its related components, ADD the new <CarouselSection /> component.

```
// Example of what HomePage.jsx might look like
import CarouselSection from './components/CarouselSection';
// ... other imports

function HomePage() {
  return (
    <> {/* or <div> */}
      {/* ALL YOUR EXISTING CODE (Nav, Search Bar, etc.) GOES HERE. DO NOT TOUCH IT. */}
      <div className="existing-search-container-etc">
        {/* ... existing search buttons ... */}
      </div>

      {/* === ADD NEW COMPONENT BELOW EXISTING CONTENT === */}
      <CarouselSection />

    </> {/* or </div> */}
  );
}
```

4. UX & STYLING FOR NEW COMPONENTS

CRITICAL: Implement these behaviors only inside the new components.

MANDATORY: Horizontal Scrolling:

Each row's card list must use display: flex; and overflow-x: auto;.

Hide the default browser scrollbar (e.g., using a new CSS file or a Tailwind plugin).

MANDATORY: Clean Styling:

Movie Cards: Use aspect-ratio: 2/3;, rounded-lg, and a transition-transform duration-200 hover:scale-105.

Row Titles: Use text-2xl font-bold text-white mb-4.

Nav Arrows: Style as rounded-full, bg-black/50, text-white, z-index: 20, position: absolute;.

MANDATORY: Drag-to-Scroll:

Implement "click and drag" scrolling on each row using onMouseDown, onMouseMove, onMouseUp, and onMouseLeave events.

MANDATORY: Overlayed Navigation Arrows:

Arrows must be invisible by default and appear only on row-hover (e.g., using group-hover:block).

Clicks must scroll the container smoothly.

MANDATORY: Hover-to-Autoscroll:

Implement the very slow auto-scroll on mouseenter.

Auto-scroll must stop on mouseleave, drag, or arrow click.

5. MANDATORY: Verification Steps

CRITICAL: Verify the existing application (header, search bar) is 100% UNCHANGED in appearance and function.

CRITICAL: Verify the new carousel section appears below the existing content.

CRITICAL: Verify the carousel section has its own dark background, while the header/search area retains its original styling.

MANDATORY: Verify all 9 Node.js API endpoints are working and return mock JSON.

MANDATORY: Verify all 9 carousels render with their correct titles and movie posters.

MANDATORY: Verify all 4 interaction models work on each row (drag, arrows, hover-scroll, hidden scrollbar).

MANDATORY: Verify the auto-scroll is slow and stops on mouse-leave, drag, or arrow click.

CRITICAL REQUIREMENT

Safety first. The implementation must be purely additive. Do not break the existing application. All new components must be isolated, data-driven, and render after the current UI, without modifying any existing files other than to add the single <CarouselSection /> component.

CRITICAL REQUIREMENTS - Netflix-Style Gradient Navigation Arrows

MANDATORY DIRECTIVE

You are an expert Frontend Developer specializing in UI Polish and Tailwind CSS.

CRITICAL: Modify the MovieCarouselRow component to replace the current "circle button" navigation arrows with full-height gradient scrims. The "Next" and "Previous" buttons should no longer look like floating circles, but rather like subtle vertical shadows on the edges of the carousel that house the arrows.

1. COMPONENT MODIFICATION TARGET

File: components/MovieCarouselRow.jsx (or wherever the carousel row is defined).

2. CSS & TAILWIND REQUIREMENTS

CRITICAL: 1. Remove Old Styles

MANDATORY: Remove the following classes from the navigation buttons:

rounded-full

bg-black/50 (or any solid background)

top-50% / translate-y centering (we will use flexbox instead).

w-8 h-8 (or fixed small dimensions).

CRITICAL: 2. Implement "Gradient Scrim" Containers

Concept: The button is now a tall, rectangular strip sitting on the edge of the carousel.

MANDATORY: Apply these base styles to BOTH left and right buttons:

absolute

top-0

bottom-0 (Full Height)

z-40 (High z-index to sit above posters)

w-12 or w-16 (Wide click area)

m-0 (No margins)

flex items-center justify-center (To vertically center the arrow icon)

opacity-0 group-hover:opacity-100 (Only visible on hover)

transition-opacity duration-300 ease-in-out

cursor-pointer

CRITICAL: 3. Specific Gradient Directions

MANDATORY: The gradient must be darker at the screen edge and fade to transparent inwards.

Left Arrow (Previous):

Position: left-0

Gradient: bg-gradient-to-r from-black/80 to-transparent

Right Arrow (Next):

Position: right-0

Gradient: bg-gradient-to-l from-black/80 to-transparent

MANDATORY: 4. Icon Styling

CRITICAL: The < and > icons (or SVGs) inside the button must remain white.

Optional: Add a hover scale effect to the icon only, not the gradient container:

Wrap the icon in a or apply directly to SVG: hover:scale-125 transition-transform duration-200.

3. EXPECTED CODE STRUCTURE (Example)

{/* Left Arrow (Previous) */}

```
<button
  className={`absolute left-0 top-0 bottom-0 z-40 w-14 bg-gradient-to-r from-black/80 to-transparent flex items-center justify-center opacity-0 group-hover:opacity-100 transition-opacity duration-300 ${!isMoved && "hidden"}`}
  onClick={() => handleClick("left")}
>
  <ChevronLeftIcon className="h-8 w-8 text-white transition-transform duration-200 hover:scale-125" />
</button>

{/* Movie Container */}
<div className="...">...</div>

{/* Right Arrow (Next) */}
<button
  className="absolute right-0 top-0 bottom-0 z-40 w-14 bg-gradient-to-l from-black/80 to-transparent flex items-center justify-center opacity-0 group-hover:opacity-100 transition-opacity duration-300"
  onClick={() => handleClick("right")}
>
  <ChevronRightIcon className="h-8 w-8 text-white transition-transform duration-200 hover:scale-125" />
</button>
```

4. MANDATORY: Verification Steps

CRITICAL: Verify the buttons extend from the very top to the very bottom of the carousel row (covering the posters vertically).

CRITICAL: Verify the "Left" button gradient is Black -> Transparent (Dark on the left edge).

CRITICAL: Verify the "Right" button gradient is Transparent -> Black (Dark on the right edge).

CRITICAL: Verify the arrow icon is perfectly centered within this gradient strip.

CRITICAL: Verify the buttons are invisible (opacity-0) until the user hovers over the row.

CRITICAL: Verify the "Previous" arrow remains hidden if the scroll is at the start (scrollLeft === 0).

CRITICAL REQUIREMENT

Visual Polish: The gradient must be smooth. Use from-black/80 (or from-gray-900/90) instead of solid black to avoid a harsh cutoff line. The goal is a subtle vignette effect that allows the user to see the arrow clearly against colorful movie posters.

CRITICAL REQUIREMENTS - Carousel Edge Fade & Dynamic Opacity

MANDATORY DIRECTIVE

You are an expert Frontend Developer specializing in React animations and CSS visual effects.

CRITICAL: Implement a sophisticated "Edge Fade" effect for the MovieCarouselRow. This involves two distinct mechanisms: visual gradient overlays at the container edges, and a dynamic, scroll-aware opacity calculation for individual movie cards. Crucially, the currently "Selected" movie must be immune to these effects.

1. CSS IMPLEMENTATION (Gradient Overlays)

MANDATORY: Update Styles (MovieCarouselRow.css or Tailwind)

CRITICAL: Do NOT use mask-image on the entire container, as this cuts off the selected card if it overlaps the edge. Use pseudo-element overlays instead.

MANDATORY: Target the .movie-list container (or the scrollable viewport).

MANDATORY: Add ::before and ::after pseudo-elements to create the fade strips.

Position: absolute, top-0, bottom-0, width approx 4rem to 6rem.

Z-Index: z-30 (Must be above standard cards but below the selected card).

Pointer Events: none (Allow clicks to pass through to cards).

MANDATORY: Gradients:

::before (Left): left-0, bg-gradient-to-r from-black to-transparent.

::after (Right): right-0, bg-gradient-to-l from-black to-transparent.

2. REACT LOGIC (Dynamic Opacity)

MANDATORY: Update Component (MovieCarouselRow.jsx)

CRITICAL: Implement a useEffect hook to handle dynamic card opacity based on scroll position.

Logic Requirements:

Event Listeners: Attach listeners to the scrollable container's scroll event and the window's resize event.

Calculation: For every movie card in the list:

Get the card's getBoundingClientRect().

Get the container's getBoundingClientRect().

Calculate the card's distance from the left and right edges of the viewable area.

Opacity Value:

If card is near the edge: Interpolate opacity between 0.3 (at edge) and 1.0 (fully visible).

If card is in the center: opacity: 1.

Performance: Ensure calculations are efficient. Use requestAnimationFrame if necessary to prevent scroll jank.

3. SELECTED CARD EXCEPTION (CRITICAL)

CRITICAL: "Selected Card" Protection Rule

MANDATORY: The logic above must NEVER fade the currently selected movie.

Identification: Use the existing logic that applies the selection ring (e.g., checking selectedMovie.id === movie.id or the presence of the ring-2 class).

CSS Override:

The selected card must have opacity: 1 !important.

The selected card must have a higher Z-Index (e.g., z-50) than the gradient overlays (::before/::after) so it appears to "pop" over the fade effect when scrolling near the edge.

React Logic Exemption: Inside the useEffect calculation loop, explicitly skip the opacity assignment for the selected card ID.

4. MANDATORY: Verification Steps

CRITICAL: Verify there are black gradient strips on the far left and far right of the carousel.

CRITICAL: Scroll the carousel. Verify that unselected cards smoothly fade to ~30% opacity as they approach the edges.

CRITICAL: Click a card near the edge to select it.

CRITICAL: VERIFY THE SELECTED CARD IS FULLY VISIBLE. It must NOT be faded, and it must appear on top of the black gradient overlay.

CRITICAL: Verify that resizing the window updates the fade calculations immediately.

CRITICAL: Verify that clicking through the gradient (on the edge) still registers clicks on the cards below (due to pointer-events: none).

CRITICAL REQUIREMENT

Outcome: A polished, Netflix-style carousel where the user focus (the selected item) is prioritized and never obscured, while peripheral items recede visually via opacity and gradients.

CRITICAL REQUIREMENTS - Responsive Scaling & Layout Improvements

MANDATORY DIRECTIVE

You are an expert Frontend Developer specializing in Responsive Design and Tailwind CSS.

CRITICAL: Update the application to scale gracefully on larger screens (Desktop, 2K, 4K). The current layout is too static; you must implement fluid sizing, responsive breakpoints, and dynamic column counting to utilize screen real estate effectively.

1. DYNAMIC COLUMN LOGIC (JavaScript)

MANDATORY: Update MovieList.jsx (or relevant list component)

CRITICAL: Modify the window width calculation logic to handle larger breakpoints (2xl).

MANDATORY: Implement the following column count logic based on window.innerWidth:

>= 1536px (2xl): Set columns to 7.

>= 1280px (xl): Set columns to 6 (or 5, depending on density preference).

>= 1024px (lg): Set columns to 4.

Default (< 1024px): Fallback to existing logic (e.g., 2 or 3).

MANDATORY: Ensure this logic runs on mount and on window.resize events.

2. FLUID CARD SIZING (CSS/Tailwind)

MANDATORY: Update MovieCarousel.css (or card styles)

CRITICAL: Replace fixed pixel widths with fluid clamp() values to allow cards to grow/shrink within reasonable bounds.

MANDATORY: Apply the following styles to the movie card container:

```
width: clamp(200px, 15vw, 280px);
min-width: 200px;
aspect-ratio: 2/3;
flex: 0 0 auto; /* Prevent squishing in flex container */
```

Reasoning: This ensures cards are never smaller than 200px but scale up to 280px on wide screens, maintaining their poster aspect ratio.

3. RESPONSIVE LAYOUT & TYPOGRAPHY (Tailwind)

MANDATORY: Update CarouselSection.jsx

CRITICAL: Increase vertical and horizontal padding on larger screens to prevent content from touching edges or feeling cramped.

MANDATORY: Update the main container className:

Base: py-8 px-12 (or existing)

XL: xl:py-10 xl:px-12

2XL: 2xl:py-12 2xl:px-16

Background: Maintain bg-gray-100 dark:bg-gray-900.

MANDATORY: Update MovieDetails.jsx (Overlay/Modal)

CRITICAL: Scale text sizes for better readability on large displays.

MANDATORY: Apply these responsive classes:

Movie Title (h2): text-3xl md:text-4xl font-bold mb-2.

Tagline/Description (h3): text-xl xl:text-2xl font-semibold.

Overview/Body (p): text-base xl:text-lg leading-7.

MANDATORY: Increase container padding:

p-6 (Base) -> md:p-8 -> xl:p-10 -> 2xl:p-12.

4. MANDATORY: Verification Steps

CRITICAL: Resize the browser window to > 1536px. Verify that the movie list displays 7 columns (if grid) or that cards scale up to 280px (if carousel).

CRITICAL: Verify that shrinking the window below 1536px but above 1280px adjusts the layout (e.g., 6 columns or slightly smaller cards).

CRITICAL: Verify that on mobile/tablet (< 1024px), the layout reverts to the standard compact view (min-width: 200px).

CRITICAL: Open the "Movie Details" view on a large screen. Verify the text is large and legible (text-lg+) and not tiny compared to the modal size.

CRITICAL: Check that padding increases on the sides of the carousel rows (2xl:px-16) to give the content "breathing room."

CRITICAL REQUIREMENT

Outcome: The application must feel "native" to large monitors, utilizing the extra space for larger imagery and clearer text, rather than just centering a small mobile layout.

CRITICAL REQUIREMENTS - Movie Hover Overlay & Details Integration

MANDATORY DIRECTIVE

You are an expert Full-Stack Developer specializing in React and Node.js.

CRITICAL: Upgrade the MovieCarouselRow components to be interactive. Users must see a title overlay when hovering over a movie card, and clicking a card must open the full "Movie Details" view (reusing the existing search result details logic).

1. BACKEND DATA ENRICHMENT (Node.js)

MANDATORY: Update MoviesController.js

CRITICAL: The carousel endpoints currently might be returning simplified objects. They must return complete movie objects to populate the details view.

MANDATORY: In your controller functions (e.g., getFeatured, getDrama, etc.), ensure the mapped response includes:

id

title

overview (Description)

releaseDate

posterPath (Original TMDb path)

posterUrl (Full URL if constructed)

voteAverage (if available)

2. COMPONENT INTERACTIVITY (React)

MANDATORY: Update App.jsx & CarouselSection.jsx

CRITICAL: Wiring the Selection Logic.

MANDATORY: In App.jsx, pass the existing setSelectedMovie (or similar state setter used for search results) down to the <CarouselSection /> component.

MANDATORY: In CarouselSection.jsx, accept this prop (e.g., onMovieSelect) and pass it down to every <MovieCarouselRow /> instance.

MANDATORY: Update MovieCarouselRow.jsx

CRITICAL: Implement the click interaction.

MANDATORY: Accept onMovieSelect as a prop.

MANDATORY: Attach an onClick handler to the movie card container.

Logic: When clicked, execute onMovieSelect(movie).

UX Constraint: Ensure that dragging the carousel does not accidentally trigger a click. (Check if e.target or mouse movement suggests a drag action vs a click).

3. HOVER OVERLAY STYLING (CSS/Tailwind)

MANDATORY: Update MovieCarousel.css (or Tailwind classes)

CRITICAL: Create a visual overlay that appears only on hover.

MANDATORY: Inside the movie card container, add a new div for the overlay.

Styling Requirements:

Position: absolute, bottom-0, left-0, right-0.

Background: bg-gradient-to-t from-black/90 via-black/60 to-transparent.

Padding: p-4.

Visibility: opacity-0 by default, group-hover:opacity-100 on hover.

Transition: transition-opacity duration-300.

Content: Inside this overlay, display the Movie Title in white, bold text (e.g., text-sm font-bold text-white).

4. DATA COMPATIBILITY

MANDATORY: Update MovieDetails.jsx

CRITICAL: Handle different data shapes.

MANDATORY: The carousel might provide posterUrl (full link), while search results might provide posterPath (partial).

Logic: Update the image rendering logic to check both:

```
const displayImage = movie.posterUrl || (movie.posterPath ?
`https://image.tmdb.org/t/p/w500${movie.posterPath}` : placeholder);
```

5. MANDATORY: Verification Steps

CRITICAL: Refresh the page. Hover over a movie in any carousel. Verify a dark gradient overlay appears at the bottom with the movie title.

CRITICAL: Click a movie in the carousel. Verify the "Movie Details" section (dropdown/modal) appears at the top, populated with the correct title, description, and release date.

CRITICAL: Verify that clicking while dragging (swipe interaction) does not open the details view (if possible to test).

CRITICAL: Check the browser console to ensure no props validation warnings occur regarding the movie object structure.

CRITICAL REQUIREMENT

Outcome: A seamless user experience where browsing (hovering) provides context, and selection (clicking) provides detail, unified across both the search results and the curated carousels.